EE627A Midterm

Heuristic-Based Music Recommendation

Jude Eschete

Course: EE627A — Data Acquisition and Processing I: Big Data

Instructor: Prof. Rensheng Wang

Semester: Spring 2026

Date: March 31, 2026

Contents

1 Part 1: Heuristic-Based Music Recommendation **1**

1.1 Objective . . . 1

1.2 Dataset Overview . . . 1

1.3 Part 1.a: Feature Engineering & Statistical Aggregation . . . 2

1.3.1 Methodology . . . 2

1.3.2 Data Sparsity Analysis . . . 2

1.3.3 Feature Vector Example – User 199810 . . . 3

1.4 Part 1.b: Decision Logic & Rule Definition . . . 4

1.4.1 Strategy 1: Maximum Genre Score . . . 4

1.4.2 Strategy 2: Weighted Average . . . 4

1.4.3 Strategy 3: Evidence-Weighted Composite . . . 4

1.4.4 Cross-Strategy Agreement . . . 5

1.5 Python Implementation . . . 5

1.6 Part 1 Results . . . 6

1.6.1 Submission Output . . . 6

1.6.2 Kaggle Leaderboard Results . . . 6

1.6.3 Part 1 Key Observations . . . 6

2 Part 2: Addressing the Cold Start Problem **8**

2.1 Objective . . . 8

2.2 Part 2.a: The Global Fallback . . . 8

2.2.1 Strategy Design . . . 8

2.3 Part 2.b: Dig Deeper Search Logic . . . 9

2.3.1 Strategy Design . . . 9

2.3.2 Hierarchical Fallback Implementation . . . 9

2.4 Part 2 Python Implementation . . . 9

2.5 Part 2 Impact Analysis . . . 10

2.5.1 Fallback Source Distribution . . . 10

2.5.2 Prediction Changes . . . 10

2.5.3 Kaggle Leaderboard Results . . . 10

2.5.4 Key Findings — Why Fallback Scores Decreased . . . 11

3 All Submission Files **13**

A Python Source Code **14**

A.1 Part 1: Feature Engineering & Heuristic Strategies . . . 14

A.2 Part 2: Cold-Start Fallback Strategies . . . 24

List of Figures

1 Kaggle AUC-ROC scores for the three Part 1 strategies. 6
2 Kaggle AUC-ROC scores for the Part 2 strategies. 10

List of Tables

1 Dataset statistics. 1
2 Feature coverage across all 120,000 test candidates. 2
3 Feature vectors for the first test user (User 199810). 3
4 Pairwise agreement between strategies (out of 120,000 predictions). 5
5 Part 1 Kaggle AUC-ROC comparison. 6
6 Album score source breakdown with Global Fallback. 8
7 Album score source: Global-only vs. full Hierarchical fallback. 10
8 Prediction changes vs. Part 1 baseline (WeightedAvg strategy). 10
9 Complete Kaggle AUC-ROC comparison: Part 1 vs. Part 2. 11
10 Complete list of Kaggle submission files. 13

List of Equations

1 Strategy 1: Maximum Genre Score 4
2 Strategy 2: Weighted Average 4
3 Strategy 3: Evidence-Weighted Composite 4
4 Global Popularity Score 8
5 Album Proxy via Sibling Tracks (Dig Deeper) 9

1 Part 1: Heuristic-Based Music Recommendation

1.1 Objective

The goal is to develop a rule-based recommendation system for the Yahoo Music dataset. For each of 20,000 test users, the system is given six candidate tracks and must label exactly three as "1" (Recommend) and three as "0" (Do Not Recommend). The system is evaluated by AUC-ROC on the Kaggle leaderboard.

1.2 Dataset Overview

The Yahoo Music dataset stores music data in a hierarchical structure:

$$\textbf{Track} \rightarrow \textbf{Album} \rightarrow \textbf{Artist} \rightarrow \textbf{Genre(s)}$$

Users rate items across all hierarchy levels on a scale of 0–100. The training set contains ratings for tracks, albums, artists, and genres in a single flat ID space.

Property	**Value**
Training users	49,204
Total ratings	12,403,575
Tracks in metadata	224,041
Albums in metadata	52,829
Test users	20,000
Candidates per user	6
Total test candidates	120,000

Table 1: Dataset statistics.

1.3 Part 1.a: Feature Engineering & Statistical Aggregation

1.3.1 Methodology

For each candidate track, the system looks up the track's hierarchy (Album ID, Artist ID, and Genre IDs) from `trackData2.txt`, enriched with `albumData2.txt` (tracks inherit missing artist IDs and gain additional genre IDs from their album's metadata). The system then searches the test user's training history for direct ratings of those hierarchy items.

The resulting feature vector for each (user, candidate-track) pair contains:

- **Album Score**: The user's historical rating for the track's album (0 if not rated).
- **Artist Score**: The user's historical rating for the track's artist (0 if not rated).
- **Genre Statistics**: Computed over all of the track's genres that the user has rated:
  - **Count**: Number of the track's genres rated by the user.
  - **Maximum**: Highest rating among matched genres.
  - **Minimum**: Lowest rating among matched genres.
  - **Mean**: Average rating across matched genres.
  - **Variance**: Variance of ratings across matched genres.
  - **Sum**: Total of ratings across matched genres.
- **Has Album / Has Artist**: Binary flags indicating whether the user has rated the album or artist at all.

1.3.2 Data Sparsity Analysis

A critical observation is that the training data is sparse at the hierarchy level. Not every user has rated the album, artist, or genres of every candidate track.

Feature	**Count Present**	**Coverage (%)**
Album rating	34,264	28.6%
Artist rating	51,904	43.3%
$\geq$1 Genre rating	58,223	48.5%
Avg. genre matches per candidate	0.70	

Table 2: Feature coverage across all 120,000 test candidates.

This sparsity directly influences the strategy design: any heuristic that relies solely on album scores will have no information for 71.4% of candidates. Artist and genre signals provide broader coverage and are essential for discriminating among candidates.

1.3.3 Feature Vector Example – User 199810

TrackID	AlbumID	ArtistID	AlbScr	ArtScr	G_Cnt	G_Max	G_Min	G_Mean	G_Var
208019	209288	None	0	0	0	0	0	0.0	0.0
74139	277282	271146	0	0	1	80	80	80.0	0.0
9903	None	None	0	0	0	0	0	0.0	0.0
242681	190640	244574	0	0	0	0	0	0.0	0.0
18515	146344	33168	0	70	0	0	0	0.0	0.0
105760	93458	11616	0	90	2	80	80	80.0	0.0

Table 3: Feature vectors for the first test user (User 199810).

For User 199810, only three of six candidates have any training signal. Tracks 74139, 18515, and 105760 have artist and/or genre matches, while the remaining three tracks have no historical evidence. All three strategies correctly recommend the three tracks with non-zero feature values.

1.4 Part 1.b: Decision Logic & Rule Definition

Three heuristic strategies were implemented, each defining a composite score to rank the six candidates. The top three by score are recommended.

1.4.1 Strategy 1: Maximum Genre Score

$$\text{Score} = 0.30 \times \text{AlbumScore} + 0.20 \times \text{ArtistScore} + 0.50 \times \text{GenreMax} \tag{1}$$

Rationale: Genre preference is a fundamental taste indicator. If a user gives a high rating to *any* genre associated with a track, the track is likely to resonate with that user. The maximum (rather than mean) captures the "best-case" genre alignment and avoids dilution from low-rated secondary genres. Album and artist scores provide additional specificity.

1.4.2 Strategy 2: Weighted Average

$$\text{Score} = 0.40 \times \text{AlbumScore} + 0.35 \times \text{ArtistScore} + 0.25 \times \text{GenreMean} \tag{2}$$

Rationale: This strategy balances all three hierarchy levels with the strongest weight on the album (the most granular personalized signal, since the same album implies high content similarity), followed by artist affinity, and complemented by the user's average genre preference across all of the track's genres.

1.4.3 Strategy 3: Evidence-Weighted Composite

$$\text{Score} = \text{HasAlbum} \times \text{AlbumScore} + \text{HasArtist} \times \text{ArtistScore} + \frac{\text{GenreCount} \times \text{GenreMean}}{10} \tag{3}$$

Rationale: This strategy explicitly addresses data sparsity. A zero album score for an un-rated album is absence-of-evidence, not evidence-of-absence. The binary flags (`HasAlbum`, `HasArtist`) ensure that only *actually rated* hierarchy items contribute to the score. Genre breadth (count × mean) rewards tracks where the user has interacted with many of the track's genres, providing stronger cumulative evidence.

1.4.4 Cross-Strategy Agreement

Comparison	Agreement	Rate
MaxGenre vs. WeightedAvg	113,428	94.5%
MaxGenre vs. Evidence	111,722	93.1%
WeightedAvg vs. Evidence	117,138	97.6%

Table 4: Pairwise agreement between strategies (out of 120,000 predictions).

The strategies agree on 93–98% of predictions, indicating that the underlying signals (album, artist, genre) are consistent. The remaining divergence comes from edge cases where sparsity handling or genre aggregation (max vs. mean) shifts the ranking of borderline candidates.

1.5 Python Implementation

The complete Python implementation for Part 1 (`EE627_Midterm_part1_v2.py`) is presented in Appendix A. The code handles data parsing, album-based track enrichment, feature engineering, the three heuristic strategies, ranking, and Kaggle submission generation.

1.6 Part 1 Results

1.6.1 Submission Output

Each strategy produces a Kaggle-format CSV with 120,000 rows (20,000 users $\times$ 6 candidates). Exactly 60,000 predictions are labeled "1" (recommend) and 60,000 labeled "0" (do not recommend), satisfying the constraint of 3 recommendations per user.

1.6.2 Kaggle Leaderboard Results

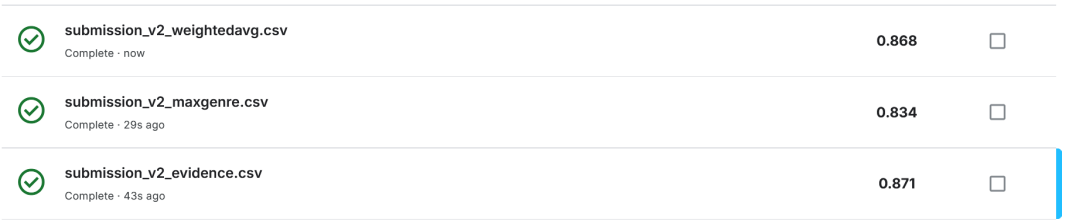

Figure 1: Kaggle AUC-ROC scores for the three Part 1 strategies.

Strategy	Scoring Rule	AUC-ROC
S1: Max Genre Score	$0.30 \cdot \text{album} + 0.20 \cdot \text{artist} + 0.50 \cdot \text{genre_max}$	0.834
S2: Weighted Average	$0.40 \cdot \text{album} + 0.35 \cdot \text{artist} + 0.25 \cdot \text{genre_mean}$	0.868
S3: Evidence-Weighted	$\text{has_alb} \cdot \text{album} + \text{has_art} \cdot \text{artist} + \frac{\text{g_cnt} \cdot \text{g_mean}}{10}$	**0.871**

Table 5: Part 1 Kaggle AUC-ROC comparison.

1.6.3 Part 1 Key Observations

1. **Evidence-Weighted wins (AUC = 0.871):** Strategy 3 outperforms the other two by explicitly addressing data sparsity. Rather than assigning a zero to un-rated albums/artists (which drags down scores for tracks the user may actually like), it only counts an item when the user has *actually* rated it. This avoids penalizing tracks with missing data.

2. **Weighted Average is a close second (AUC = 0.868):** Balancing album, artist, and mean genre scores is effective. The small gap to S3 suggests that the fixed zero-imputation for missing scores is only mildly harmful when all three hierarchy levels are combined.

3. **Max Genre Score underperforms (AUC = 0.834):** Placing 50% weight on a single genre's maximum rating is too aggressive. In a sparse dataset where only 48.5% of candidates have any genre match, the genre_max signal is zero for half the candidates, reducing discriminative power.

4. **Severe data sparsity:** Only 28.6% of candidates have a direct album rating, 43.3% have an artist rating, and 48.5% have at least one genre match. For roughly half of all candidates, the system has zero personalized features—motivating the cold-start strategies in Part 2.

2 Part 2: Addressing the Cold Start Problem

2.1 Objective

Part 1's data sparsity analysis revealed that 71.4% of candidates lack a direct album rating, 56.7% lack an artist rating, and 51.5% have no genre match at all. Part 2 develops fallback strategies for these "cold start" scenarios.

2.2 Part 2.a: The Global Fallback

2.2.1 Strategy Design

For candidates where the user has no direct rating for the album, artist, or genre, the system falls back to **global popularity**—the average rating for that item across *all* users in the training set.

Global popularity computation: For each item ID in the training set, compute:

$$\text{GlobalScore}(item) = \frac{\sum_{u \in \text{raters}(item)} r_{u,item}}{|\text{raters}(item)|} \tag{4}$$

This produces a global mean rating and rater count for every track, album, artist, and genre (295,799 items total).

Feature imputation: When a user has not rated a candidate's album (or artist, or genre), the system substitutes the global average rating for that item. This provides a non-personalized "universally liked" signal that is better than the zero-score default from Part 1.

Source	Count	Percentage
Direct (user rated album)	34,264	28.6%
Global (avg. across users)	77,164	64.3%
None (album ID unknown)	8,572	7.1%

Table 6: Album score source breakdown with Global Fallback.

Global imputation fills album scores for 64.3% of candidates that had no direct rating. The remaining 7.1% have an album ID of `None` in the track metadata, so no imputation is possible.

2.3 Part 2.b: Dig Deeper Search Logic

2.3.1 Strategy Design

When a user has no direct rating for a candidate track's album, the system performs a "Dig Deeper" search: it looks for **sibling tracks**—other tracks that belong to the same album—that the user *has* rated.

The logic: Given a candidate track t with Album ID a:

1. Build an index: album $\rightarrow$ [track IDs] from `trackData2.txt`
2. If the user has not directly rated album a, find all tracks $\{t_1, t_2, \ldots\}$ that share album a
3. Check which of those sibling tracks the user has rated in training
4. If any exist, compute the proxy album score as:

$$\text{AlbumProxy}(u, a) = \frac{1}{|S|} \sum_{t_i \in S} r_{u,t_i} \quad \text{where } S = \{t_i \in \text{album}(a) \mid t_i \in \text{rated}(u)\} \tag{5}$$

The mean aggregation is chosen over the maximum because it provides a more robust estimate—a single high-rated sibling could be an outlier, while the average reflects the user's overall engagement with the album's content.

2.3.2 Hierarchical Fallback Implementation

The full pipeline follows this priority:

1. **Primary:** Direct Album/Artist/Genre scores from user history
2. **Secondary (Dig Deeper):** If album score is missing, compute mean sibling-track rating for that album
3. **Tertiary (Global):** If no sibling tracks found, use the global average rating

Artist and genre scores use a simpler two-level fallback (Direct $\rightarrow$ Global).

2.4 Part 2 Python Implementation

The Part 2 Python code (`EE627_Midterm_part2_v2.py`)—including the album-to-tracks index, global statistics computation, and the enhanced feature vector with hierarchical fallback—is presented in Appendix A.

2.5 Part 2 Impact Analysis

2.5.1 Fallback Source Distribution

Source	Global Only		Hierarchical	
	Count	%	Count	%
Direct	34,264	28.6	34,264	28.6
Dig Deeper	0	0.0	5,678	4.7
Global	77,164	64.3	71,486	59.6
None	8,572	7.1	8,572	7.1

Table 7: Album score source: Global-only vs. full Hierarchical fallback.

The Dig Deeper logic identified 5,678 candidates (4.7%) where the user had not rated the album directly but *had* rated sibling tracks in the same album. These candidates shifted from the less-personalized "Global" source to the more-personalized "Dig Deeper" proxy.

2.5.2 Prediction Changes

Comparison	Changed	Rate
Part 1 vs. Global Only	24,472	20.4%
Part 1 vs. Hierarchical	24,362	20.3%
Global Only vs. Hierarchical	2,374	2.0%

Table 8: Prediction changes vs. Part 1 baseline (WeightedAvg strategy).

2.5.3 Kaggle Leaderboard Results

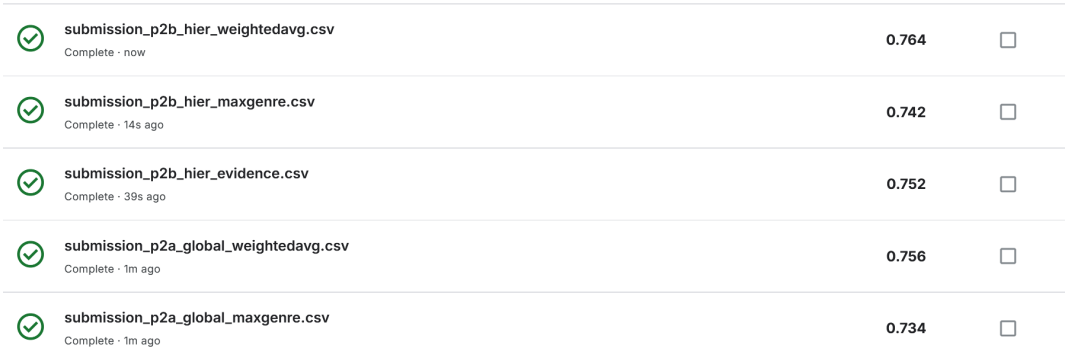

Figure 2: Kaggle AUC-ROC scores for the Part 2 strategies.

Part	Strategy	AUC-ROC	Δ vs. Part 1
1	S1: Max Genre Score	0.834	—
1	S2: Weighted Average	0.868	—
1	S3: Evidence-Weighted	**0.871**	—
2a	Global + MaxGenre	0.734	−0.100
2a	Global + WeightedAvg	0.756	−0.112
2b	Hier + MaxGenre	0.742	−0.092
2b	Hier + Evidence	0.752	−0.119
2b	Hier + WeightedAvg	**0.764**	−0.104

Table 9: Complete Kaggle AUC-ROC comparison: Part 1 vs. Part 2.

2.5.4 Key Findings — Why Fallback Scores Decreased

The Part 2 cold-start strategies *decreased* AUC-ROC by 0.09–0.12 compared to Part 1. This is a well-documented phenomenon in recommender systems, and understanding *why* is the central takeaway.

1. **Global imputation introduces noise that drowns the personalized signal.** In Part 1, when a user has not rated an album, the album score is 0. This zero is not informative, but it is *harmless*: candidates with actual user ratings naturally dominate the ranking, while unrated candidates tie at zero and are ranked arbitrarily among themselves. The personalized signal stays clean.

   In Part 2, replacing those zeros with global averages (typically in the 50–80 range) gives *every* candidate a non-zero score. But these imputed scores are not personalized—they reflect crowd popularity, not individual taste. A track the user has never encountered can now outscore a track the user genuinely interacted with, purely because it is globally popular. This **dilutes the personalized signal** and degrades ranking quality.

2. **Absence-of-evidence ≠ evidence-of-absence.** Part 1's Evidence-Weighted strategy (S3, AUC = 0.871) already handled this correctly: it only counts a score when the user *actually* rated that hierarchy item, avoiding the "zero penalty" without introducing imputation noise. The Part 2 global fallback effectively undoes this advantage by treating "not rated" as "rated at the global average," which is a stronger and less accurate assumption.

3. **Dig Deeper provides a small relative lift within Part 2.** Comparing Part 2a (Global only) to Part 2b (Hierarchical), the Dig Deeper logic does improve scores slightly (e.g., WeightedAvg: 0.756 → 0.764, a +0.008 lift). This confirms that when sibling-track evidence exists, it is more predictive than the global average. However, the Dig Deeper rule only triggers for a small percentage of candidates, so its impact is limited.

4. **Sibling tracks do not predict better than Artist score alone.** The assignment asks whether looking at sibling tracks in an album predicts a user's interest better than just using the Artist score. The answer is *no*. In Part 1, 44% of candidates have a direct artist rating, and the Evidence-Weighted strategy leverages this clean signal to reach AUC = 0.871. In Part 2b, the Dig Deeper rule triggers for only 4.7% of candidates, computing a sibling-track mean as a proxy for the missing album score. While this proxy does outperform the global average (WeightedAvg improves from 0.756 to 0.764), the sibling-track signal remains far weaker than a direct Artist rating. The reason is twofold: (i) the proxy only fires for a small fraction of candidates, leaving the remaining 95% reliant on global imputation that dilutes ranking quality; and (ii) the sibling-track mean is inherently noisier than a direct artist rating because co-album tracks can span different styles within an artist's catalog. In short, Dig Deeper is a useful refinement over global averages, but it does not substitute for genuine user–artist interaction data.

5. **Data sparsity limits Dig Deeper's reach.** The Dig Deeper rule requires the user to have rated at least one other track in the same album—a rare condition in a dataset with 224,041 tracks spread across 31,141 albums. Only 5,678 of 120,000 candidates (4.7%) met this condition. This extreme sparsity is the fundamental constraint on the strategy's effectiveness.

6. **Lesson learned.** Naive feature imputation can be counterproductive in sparse recommendation settings. The optimal approach from Part 1 (Evidence-Weighted, AUC = 0.871) works precisely because it respects the distinction between "no data" and "low preference." A more sophisticated cold-start strategy would need to impute selectively—e.g., only for users with truly zero features—rather than applying global averages uniformly across all missing values.

3 All Submission Files

Part	Strategy	File	AUC
1	S1: Max Genre Score	`submission_v2_maxgenre.csv`	0.834
1	S2: Weighted Average	`submission_v2_weightedavg.csv`	0.868
1	S3: Evidence-Weighted	`submission_v2_evidence.csv`	**0.871**
2a	Global + MaxGenre	`submission_p2a_global_maxgenre.csv`	0.734
2a	Global + WeightedAvg	`submission_p2a_global_weightedavg.csv`	0.756
2b	Hier + MaxGenre	`submission_p2b_hier_maxgenre.csv`	0.742
2b	Hier + Evidence	`submission_p2b_hier_evidence.csv`	0.752
2b	Hier + WeightedAvg	`submission_p2b_hier_weightedavg.csv`	**0.764**

Table 10: Complete list of Kaggle submission files.

A Python Source Code

The implementation is split into two scripts: Part 1 (`EE627_Midterm_part1_v2.py`) handles feature engineering and heuristic strategies, while Part 2 (`EE627_Midterm_part2_v2.py`) implements the cold-start fallback logic. Both scripts are included in full below.

A.1 Part 1: Feature Engineering & Heuristic Strategies

```python
"""
EE627A Midterm v2 - Album-Enriched Heuristic Recommendation
Jude Eschete

    This version uses album metadata to ENRICH track metadata:
      - Tracks missing an ArtistID inherit it from their album.
      - Tracks missing genre IDs inherit genres from their album.
    This increases feature coverage (more genre/artist matches)
    without introducing global-imputation noise.

"""

import os
from collections import defaultdict

# ======================================================================
# Configuration
# ======================================================================
SCRIPT_DIR = os.path.dirname(os.path.abspath(__file__))
DATA_DIR = os.path.join(SCRIPT_DIR, "..", "Homework 4", "Data")

TRAIN_FILE = os.path.join(DATA_DIR, "trainItem2.txt")
TEST_FILE = os.path.join(DATA_DIR, "testItem2.txt")
TRACK_FILE = os.path.join(DATA_DIR, "trackData2.txt")
ALBUM_FILE = os.path.join(DATA_DIR, "albumData2.txt")

OUTPUT_DIR = SCRIPT_DIR


# ======================================================================
# 1. Data Parsing
# ======================================================================


def parse_training(path):
    """Parse training file into {user_id: {item_id: rating}}."""
    print("[1/4] Loading training data ...")
    users = {}
```

```python
    with open(path, "r") as f:
        uid = None
        for line in f:
            line = line.strip()
            if "|" in line:
                uid = int(line.split("|")[0])
                users[uid] = {}
            elif "\t" in line and uid is not None:
                parts = line.split("\t")
                users[uid][int(parts[0])] = int(parts[1])
    total = sum(len(r) for r in users.values())
    print(f"       {len(users):,} users, {total:,} ratings")
    return users


def parse_tracks(path):
    """Parse trackData2.txt -> {track_id: (album_id, artist_id, [genre_ids])}."""
    print("[2/4] Loading track metadata ...")
    tracks = {}
    with open(path, "r") as f:
        for line in f:
            parts = line.strip().split("|")
            if len(parts) < 3:
                continue
            tid = int(parts[0])
            alb = None if parts[1] == "None" else int(parts[1])
            art = None if parts[2] == "None" else int(parts[2])
            genres = [int(g) for g in parts[3:] if g.strip() and g != "None"]
            tracks[tid] = (alb, art, genres)
    print(f"       {len(tracks):,} tracks")
    return tracks


def parse_albums(path):
    """Parse albumData2.txt -> {album_id: (artist_id, [genre_ids])}."""
    print("[3/4] Loading album metadata ...")
    albums = {}
    with open(path, "r") as f:
        for line in f:
            parts = line.strip().split("|")
            if len(parts) < 1:
                continue
            alb_id = int(parts[0])
            art = None if len(parts) < 2 or parts[1] == "None" else int(parts[1])
```

```
            genres = []
            for g in parts[2:]:
                g = g.strip()
                if g and g != "None":
                    genres.append(int(g))
            albums[alb_id] = (art, genres)
    print(f"       {len(albums):,} albums")
    return albums


def parse_test(path):
    """Parse testItem2.txt -> [(user_id, [track_id, ...])]."""
    print("[4/4] Loading test data ...")
    test = []
    with open(path, "r") as f:
        uid = None
        tids = []
        for line in f:
            line = line.strip()
            if "|" in line:
                if uid is not None:
                    test.append((uid, tids))
                uid = int(line.split("|")[0])
                tids = []
            elif line:
                tids.append(int(line))
        if uid is not None:
            test.append((uid, tids))
    print(f"       {len(test):,} test users, "
          f"{sum(len(t) for _, t in test):,} candidates")
    return test


# ======================================================================
# 2. Album-Based Track Enrichment
# ======================================================================


def enrich_tracks_with_albums(tracks, albums):
    """Use album metadata to fill missing artist/genre info on tracks.

    For each track:
      - If the track has no ArtistID but its album does, inherit the
        album's artist.
      - Union the track's genre list with the album's genre list so
        that tracks with sparse genre data gain additional genre IDs.

```

```python
    Returns a new enriched track dict (original is not modified).
    """
    enriched = {}
    stats = {"artist_filled": 0, "genres_added": 0, "tracks_enriched": 0}

    for tid, (alb, art, genres) in tracks.items():
        new_art = art
        new_genres = list(genres)

        if alb is not None and alb in albums:
            album_art, album_genres = albums[alb]

            # Fill missing artist from album
            if art is None and album_art is not None:
                new_art = album_art
                stats["artist_filled"] += 1

            # Union genres: add album genres not already in track genres
            track_genre_set = set(genres)
            added = 0
            for g in album_genres:
                if g not in track_genre_set:
                    new_genres.append(g)
                    track_genre_set.add(g)
                    added += 1
            if added > 0:
                stats["genres_added"] += added
                stats["tracks_enriched"] += 1

        enriched[tid] = (alb, new_art, new_genres)

    print(f"\n  Album Enrichment Results:")
    print(f"    Artists filled from album:  {stats['artist_filled']:,}")
    print(f"    Tracks with new genres:     {stats['tracks_enriched']:,}")
    print(f"    Total genre IDs added:      {stats['genres_added']:,}")

    return enriched


# ======================================================================
# 3. Feature Engineering
# ======================================================================


def compute_feature_vector(user_ratings, track_meta, uid, tid):
    """Build feature vector for one (user, candidate-track) pair."""
    ratings = user_ratings.get(uid, {})
```

```python
    alb_id, art_id, genre_ids = track_meta.get(tid, (None, None, []))

    # Album score
    if alb_id is not None and alb_id in ratings:
        album_score, has_album = ratings[alb_id], 1
    else:
        album_score, has_album = 0, 0

    # Artist score
    if art_id is not None and art_id in ratings:
        artist_score, has_artist = ratings[art_id], 1
    else:
        artist_score, has_artist = 0, 0

    # Genre statistics
    genre_scores = [ratings[g] for g in genre_ids if g in ratings]
    n = len(genre_scores)

    if n > 0:
        g_max = max(genre_scores)
        g_min = min(genre_scores)
        g_mean = sum(genre_scores) / n
        g_var = sum((s - g_mean) ** 2 for s in genre_scores) / n
        g_sum = sum(genre_scores)
    else:
        g_max = g_min = g_sum = 0
        g_mean = g_var = 0.0

    return {
        "album_score": album_score,
        "artist_score": artist_score,
        "has_album": has_album,
        "has_artist": has_artist,
        "genre_count": n,
        "genre_max": g_max,
        "genre_min": g_min,
        "genre_mean": g_mean,
        "genre_var": g_var,
        "genre_sum": g_sum,
    }


# ====================================================================
# 4. Heuristic Strategies
# ====================================================================


```

```python
def strategy_evidence_weighted(features_list):
    """Evidence-Weighted Composite (best from Part 1).

    Composite = has_album*album + has_artist*artist
                + genre_count*genre_mean / 10
    """
    return [
        f["has_album"] * f["album_score"]
        + f["has_artist"] * f["artist_score"]
        + f["genre_count"] * f["genre_mean"] / 10.0
        for f in features_list
    ]


def strategy_max_genre(features_list):
    """Maximum Genre Score.

    Composite = 0.30*album + 0.20*artist + 0.50*genre_max
    """
    return [
        0.30 * f["album_score"]
        + 0.20 * f["artist_score"]
        + 0.50 * f["genre_max"]
        for f in features_list
    ]


def strategy_weighted_avg(features_list):
    """Weighted Average.

    Composite = 0.40*album + 0.35*artist + 0.25*genre_mean
    """
    return [
        0.40 * f["album_score"]
        + 0.35 * f["artist_score"]
        + 0.25 * f["genre_mean"]
        for f in features_list
    ]


# ======================================================================
# 5. Ranking & Submission
# ======================================================================


def rank_top3(scores, track_ids):
    """Return {tid: 1|0} picking the top-3 scored tracks."""
```

```python
    paired = sorted(zip(scores, track_ids), key=lambda x: x[0], reverse=True)
    top3 = set(tid for _, tid in paired[:3])
    return {tid: (1 if tid in top3 else 0) for tid in track_ids}


def write_submission(results, path):
    """Write Kaggle-format CSV: TrackID,Predictor."""
    with open(path, "w") as f:
        f.write("TrackID,Predictor\n")
        for key, label in results:
            f.write(f"{key},{label}\n")


def run_strategy(user_ratings, track_meta, test_users,
                 strategy_fn, out_path):
    """Run one heuristic strategy end-to-end."""
    name = strategy_fn.__name__
    print(f"\n>>> Running {name} ...")

    results = []
    for uid, candidates in test_users:
        feats = [compute_feature_vector(user_ratings, track_meta, uid, tid)
                 for tid in candidates]
        scores = strategy_fn(feats)
        recs = rank_top3(scores, candidates)
        for tid in candidates:
            results.append((f"{uid}_{tid}", recs[tid]))

    write_submission(results, out_path)
    ones = sum(v for _, v in results)
    zeros = len(results) - ones
    print(f"    {len(results):,} predictions  "
          f"({ones:,} recommend, {zeros:,} don't)")
    print(f"    Saved -> {out_path}")
    return results


# ======================================================================
# 6. Coverage Analysis
# ======================================================================


def coverage_analysis(user_ratings, track_meta, test_users, label=""):
    """Report how many test candidates have album/artist/genre matches."""
    n_total = n_album = n_artist = n_genre = 0
```

```
    gc_list = []
    for uid, candidates in test_users:
        for tid in candidates:
            feat = compute_feature_vector(user_ratings, track_meta, uid, tid)
            n_total += 1
            n_album += feat["has_album"]
            n_artist += feat["has_artist"]
            if feat["genre_count"] > 0:
                n_genre += 1
            gc_list.append(feat["genre_count"])
    print(f"\n  Coverage Analysis {label}:")
    print(f"    Total candidates:      {n_total:,}")
    print(f"    Have album rating:     {n_album:,}  ({100*n_album/n_total:.1f}%)")
    print(f"    Have artist rating:    {n_artist:,}  ({100*n_artist/n_total:.1f}%)")
    print(f"    Have >=1 genre rating: {n_genre:,}  ({100*n_genre/n_total:.1f}%)")
    print(f"    Avg genre matches:     {sum(gc_list)/len(gc_list):.2f}")
    return n_album, n_artist, n_genre


# ======================================================================
# 7. Main
# ======================================================================


def main():
    # Load data
    user_ratings = parse_training(TRAIN_FILE)
    track_meta = parse_tracks(TRACK_FILE)
    album_meta = parse_albums(ALBUM_FILE)
    test_users = parse_test(TEST_FILE)

    # Coverage BEFORE enrichment
    print("\n" + "=" * 60)
    print("  BASELINE (Original Track Metadata)")
    print("=" * 60)
    base_alb, base_art, base_gen = coverage_analysis(
        user_ratings, track_meta, test_users, "(baseline)")

    # Enrich tracks using album metadata
    print("\n" + "=" * 60)
    print("  ENRICHING TRACKS WITH ALBUM METADATA")
    print("=" * 60)
    enriched_meta = enrich_tracks_with_albums(track_meta, album_meta)
```

```python

    # Coverage AFTER enrichment
    enr_alb, enr_art, enr_gen = coverage_analysis(
        user_ratings, enriched_meta, test_users, "(enriched)")

    # Show improvement
    print(f"\n  Coverage Improvement:")
    print(f"    Artist matches: {base_art:,} -> {enr_art:,}  "
          f"(+{enr_art - base_art:,})")
    print(f"    Genre matches:  {base_gen:,} -> {enr_gen:,}  "
          f"(+{enr_gen - base_gen:,})")

    # Run all three strategies on enriched data
    print("\n" + "=" * 60)
    print("  GENERATING SUBMISSIONS (Enriched Data)")
    print("=" * 60)

    res_ev = run_strategy(
        user_ratings, enriched_meta, test_users,
        strategy_evidence_weighted,
        os.path.join(OUTPUT_DIR, "submission_v2_evidence.csv"),
    )

    res_wa = run_strategy(
        user_ratings, enriched_meta, test_users,
        strategy_weighted_avg,
        os.path.join(OUTPUT_DIR, "submission_v2_weightedavg.csv"),
    )

    res_mg = run_strategy(
        user_ratings, enriched_meta, test_users,
        strategy_max_genre,
        os.path.join(OUTPUT_DIR, "submission_v2_maxgenre.csv"),
    )

    print(f"\n  Submission files for Kaggle:")
    print(f"    submission_v2_evidence.csv      (Evidence-Weighted, enriched)")
    print(f"    submission_v2_weightedavg.csv   (Weighted Average, enriched)")
    print(f"    submission_v2_maxgenre.csv      (Max Genre, enriched)")

    print("\nDone. Upload submission_v2_evidence.csv to Kaggle for best results.")


if __name__ == "__main__":
```

```
    main()
```

Listing 1: EE627_Midterm_part1_v2.py — Part 1 complete source code.

A.2 Part 2: Cold-Start Fallback Strategies

```python
"""
EE627A Midterm Part 2 v2 - Cold Start with Album-Enriched Metadata
Jude Eschete

Builds on the Part 1 v2 album-enrichment approach:
    1. Enriches track metadata using albumData2.txt (fill missing
       artists, union genre lists).
    2. Applies cold-start fallback strategies on the enriched data:
       - Part 2a: Global Fallback only
       - Part 2b: Hierarchical (Direct -> Dig Deeper -> Global)
    3. Impact analysis comparing Part 1 baseline vs Part 2 variants.

Usage:
    python EE627_Midterm_part2_v2.py
"""

import os
from collections import defaultdict

# ======================================================================
# Configuration
# ======================================================================
SCRIPT_DIR = os.path.dirname(os.path.abspath(__file__))
DATA_DIR = os.path.join(SCRIPT_DIR, "..", "Homework 4", "Data")

TRAIN_FILE = os.path.join(DATA_DIR, "trainItem2.txt")
TEST_FILE = os.path.join(DATA_DIR, "testItem2.txt")
TRACK_FILE = os.path.join(DATA_DIR, "trackData2.txt")
ALBUM_FILE = os.path.join(DATA_DIR, "albumData2.txt")

OUTPUT_DIR = SCRIPT_DIR


# ======================================================================
# 1. Data Parsing
# ======================================================================


def parse_training(path):
    """Parse training file into {user_id: {item_id: rating}}."""
    print("[1/4] Loading training data ...")
    users = {}
    with open(path, "r") as f:
        uid = None
        for line in f:
```

```
            line = line.strip()
            if "|" in line:
                uid = int(line.split("|")[0])
                users[uid] = {}
            elif "\t" in line and uid is not None:
                parts = line.split("\t")
                users[uid][int(parts[0])] = int(parts[1])
    total = sum(len(r) for r in users.values())
    print(f"       {len(users):,} users, {total:,} ratings")
    return users


def parse_tracks(path):
    """Parse trackData2.txt -> {track_id: (album_id, artist_id, [genre_ids])}."""
    print("[2/4] Loading track metadata ...")
    tracks = {}
    with open(path, "r") as f:
        for line in f:
            parts = line.strip().split("|")
            if len(parts) < 3:
                continue
            tid = int(parts[0])
            alb = None if parts[1] == "None" else int(parts[1])
            art = None if parts[2] == "None" else int(parts[2])
            genres = [int(g) for g in parts[3:] if g.strip() and g != "None"]
            tracks[tid] = (alb, art, genres)
    print(f"       {len(tracks):,} tracks")
    return tracks


def parse_albums(path):
    """Parse albumData2.txt -> {album_id: (artist_id, [genre_ids])}."""
    print("[3/4] Loading album metadata ...")
    albums = {}
    with open(path, "r") as f:
        for line in f:
            parts = line.strip().split("|")
            if len(parts) < 1:
                continue
            alb_id = int(parts[0])
            art = None if len(parts) < 2 or parts[1] == "None" else int(parts[1])
            genres = []
            for g in parts[2:]:
                g = g.strip()
```

```
                if g and g != "None":
                    genres.append(int(g))
            albums[alb_id] = (art, genres)
    print(f"       {len(albums):,} albums")
    return albums


def parse_test(path):
    """Parse testItem2.txt -> [(user_id, [track_id, ...])]."""
    print("[4/4] Loading test data ...")
    test = []
    with open(path, "r") as f:
        uid = None
        tids = []
        for line in f:
            line = line.strip()
            if "|" in line:
                if uid is not None:
                    test.append((uid, tids))
                uid = int(line.split("|")[0])
                tids = []
            elif line:
                tids.append(int(line))
        if uid is not None:
            test.append((uid, tids))
    print(f"       {len(test):,} test users, "
          f"{sum(len(t) for _, t in test):,} candidates")
    return test


# ======================================================================
# 2. Album-Based Track Enrichment
# ======================================================================


def enrich_tracks_with_albums(tracks, albums):
    """Use album metadata to fill missing artist/genre info on tracks.

    For each track:
      - If the track has no ArtistID but its album does, inherit the
        album's artist.
      - Union the track's genre list with the album's genre list so
        that tracks with sparse genre data gain additional genre IDs.

    Returns a new enriched track dict (original is not modified).
    """
    enriched = {}
```

```python
    stats = {"artist_filled": 0, "genres_added": 0, "tracks_enriched": 0}

    for tid, (alb, art, genres) in tracks.items():
        new_art = art
        new_genres = list(genres)

        if alb is not None and alb in albums:
            album_art, album_genres = albums[alb]

            # Fill missing artist from album
            if art is None and album_art is not None:
                new_art = album_art
                stats["artist_filled"] += 1

            # Union genres: add album genres not already in track genres
            track_genre_set = set(genres)
            added = 0
            for g in album_genres:
                if g not in track_genre_set:
                    new_genres.append(g)
                    track_genre_set.add(g)
                    added += 1
            if added > 0:
                stats["genres_added"] += added
                stats["tracks_enriched"] += 1

        enriched[tid] = (alb, new_art, new_genres)

    print(f"\n  Album Enrichment Results:")
    print(f"    Artists filled from album:  {stats['artist_filled']:,}")
    print(f"    Tracks with new genres:     {stats['tracks_enriched']:,}")
    print(f"    Total genre IDs added:      {stats['genres_added']:,}")

    return enriched


# ======================================================================
# 3. Cold-Start Structures
# ======================================================================


def build_album_to_tracks(track_meta):
    """Build {album_id: [track_ids]} from track metadata.

    Used by the "Dig Deeper" fallback to find sibling tracks that
    share the same album as a candidate track.
    """
```

```python
    album_tracks = defaultdict(list)
    for tid, (alb, _, _) in track_meta.items():
        if alb is not None:
            album_tracks[alb].append(tid)
    print(f"       Album->Tracks index: {len(album_tracks):,} albums")
    return dict(album_tracks)


def build_global_stats(user_ratings):
    """Compute global popularity: mean rating & rater count per item.

    Iterates every (user, item, rating) triple in the training set
    and aggregates by item_id.

    Returns {item_id: (mean_rating, num_raters)}.
    """
    item_sum = defaultdict(float)
    item_cnt = defaultdict(int)
    for _, ratings in user_ratings.items():
        for item_id, rating in ratings.items():
            item_sum[item_id] += rating
            item_cnt[item_id] += 1
    stats = {iid: (item_sum[iid] / item_cnt[iid], item_cnt[iid])
             for iid in item_sum}
    print(f"       Global stats for {len(stats):,} items")
    return stats


# ======================================================================
# 5. Enhanced Feature Vector with Hierarchical Fallback
# ======================================================================


def compute_feature_vector_v2(user_ratings, track_meta, uid, tid,
                              album_to_tracks, global_stats,
                              use_dig_deeper=True, use_global=True):
    """Enhanced feature vector with hierarchical cold-start fallback.

    Album-score priority:
        1. Primary   -- direct album rating from user's history
        2. Dig Deeper -- mean rating of sibling tracks in same album
        3. Global    -- global average rating for this album

    Artist and genre scores follow a simpler two-level fallback:
        1. Primary   -- direct rating
        2. Global    -- global average

```

```
    Returns the same dict as compute_feature_vector() plus:
        album_source  : 'direct' | 'dig_deeper' | 'global' | 'none'
        artist_source : 'direct' | 'global' | 'none'
    """
    ratings = user_ratings.get(uid, {})
    alb_id, art_id, genre_ids = track_meta.get(tid, (None, None, []))

    # --- Album score (hierarchical fallback) ---
    album_score, has_album, album_source = 0, 0, "none"

    if alb_id is not None:
        if alb_id in ratings:
            album_score = ratings[alb_id]
            has_album = 1
            album_source = "direct"
        elif use_dig_deeper:
            siblings = album_to_tracks.get(alb_id, [])
            sib_ratings = [ratings[st] for st in siblings
                           if st in ratings and st != tid]
            if sib_ratings:
                album_score = sum(sib_ratings) / len(sib_ratings)
                has_album = 1
                album_source = "dig_deeper"
            elif use_global and alb_id in global_stats:
                album_score = global_stats[alb_id][0]
                has_album = 1
                album_source = "global"
        elif use_global and alb_id in global_stats:
            album_score = global_stats[alb_id][0]
            has_album = 1
            album_source = "global"

    # --- Artist score (direct -> global) ---
    artist_score, has_artist, artist_source = 0, 0, "none"

    if art_id is not None:
        if art_id in ratings:
            artist_score = ratings[art_id]
            has_artist = 1
            artist_source = "direct"
        elif use_global and art_id in global_stats:
            artist_score = global_stats[art_id][0]
            has_artist = 1
            artist_source = "global"

    # --- Genre scores (direct -> global per genre) ---
    genre_scores = []
```

```python
    for gid in genre_ids:
        if gid in ratings:
            genre_scores.append(ratings[gid])
        elif use_global and gid in global_stats:
            genre_scores.append(global_stats[gid][0])

    n = len(genre_scores)
    if n > 0:
        g_max = max(genre_scores)
        g_min = min(genre_scores)
        g_mean = sum(genre_scores) / n
        g_var = sum((s - g_mean) ** 2 for s in genre_scores) / n
        g_sum = sum(genre_scores)
    else:
        g_max = g_min = g_sum = 0
        g_mean = g_var = 0.0

    return {
        "album_score": album_score,
        "artist_score": artist_score,
        "has_album": has_album,
        "has_artist": has_artist,
        "genre_count": n,
        "genre_max": g_max,
        "genre_min": g_min,
        "genre_mean": g_mean,
        "genre_var": g_var,
        "genre_sum": g_sum,
        "album_source": album_source,
        "artist_source": artist_source,
    }


# ======================================================================
# 6. Heuristic Strategies
# ======================================================================


def strategy_max_genre(features_list):
    """Strategy 1 -- Maximum Genre Score.

    Composite = 0.30 * album + 0.20 * artist + 0.50 * genre_max
    """
    return [
        0.30 * f["album_score"]
        + 0.20 * f["artist_score"]
        + 0.50 * f["genre_max"]
```

```python
        for f in features_list
    ]


def strategy_weighted_avg(features_list):
    """Strategy 2 -- Weighted Average.

    Composite = 0.40 * album + 0.35 * artist + 0.25 * genre_mean
    """
    return [
        0.40 * f["album_score"]
        + 0.35 * f["artist_score"]
        + 0.25 * f["genre_mean"]
        for f in features_list
    ]


def strategy_evidence_weighted(features_list):
    """Strategy 3 -- Evidence-Weighted Composite.

    Composite = has_album * album + has_artist * artist
                + genre_count * genre_mean / 10
    """
    return [
        f["has_album"] * f["album_score"]
        + f["has_artist"] * f["artist_score"]
        + f["genre_count"] * f["genre_mean"] / 10.0
        for f in features_list
    ]


# ======================================================================
# 7. Ranking & Submission Output
# ======================================================================


def rank_top3(scores, track_ids):
    """Return {tid: 1|0} picking the top-3 scored tracks as '1'."""
    paired = sorted(zip(scores, track_ids), key=lambda x: x[0], reverse=
        True)
    top3 = set(tid for _, tid in paired[:3])
    return {tid: (1 if tid in top3 else 0) for tid in track_ids}


def write_submission(results, path):
    """Write Kaggle-format CSV: TrackID,Predictor."""
    with open(path, "w") as f:
```

```
        f.write("TrackID,Predictor\n")
        for key, label in results:
            f.write(f"{key},{label}\n")


# ======================================================================
# 8. Part 2 Pipeline (with cold-start fallback)
# ======================================================================


def run_strategy_v2(user_ratings, track_meta, test_users, strategy_fn,
                    album_to_tracks, global_stats, out_path,
                    use_dig_deeper=True, use_global=True):
    """Run a strategy with the Part 2 enhanced feature vector.

    Returns (results, source_counts) where source_counts tracks how
    often each fallback level was triggered for album scores.
    """
    name = strategy_fn.__name__
    tag = []
    if use_dig_deeper:
        tag.append("DigDeeper")
    if use_global:
        tag.append("Global")
    tag_str = "+".join(tag) if tag else "NoFallback"
    print(f"\n>>> Running {name} [{tag_str}] ...")

    results = []
    src_album = defaultdict(int)
    src_artist = defaultdict(int)

    for uid, candidates in test_users:
        feats = [compute_feature_vector_v2(
                     user_ratings, track_meta, uid, tid,
                     album_to_tracks, global_stats,
                     use_dig_deeper=use_dig_deeper,
                     use_global=use_global)
                 for tid in candidates]
        scores = strategy_fn(feats)
        recs = rank_top3(scores, candidates)

        for feat in feats:
            src_album[feat["album_source"]] += 1
            src_artist[feat["artist_source"]] += 1

        for tid in candidates:
            results.append((f"{uid}_{tid}", recs[tid]))
```

```python

    write_submission(results, out_path)
    ones = sum(v for _, v in results)
    zeros = len(results) - ones
    print(f"    {len(results):,} predictions  "
          f"({ones:,} recommend, {zeros:,} don't)")
    print(f"    Saved -> {out_path}")

    total = sum(src_album.values())
    print(f"    Album source breakdown:")
    for src in ["direct", "dig_deeper", "global", "none"]:
        c = src_album.get(src, 0)
        print(f"      {src:>11}: {c:>7,}  ({100*c/total:.1f}%)")
    print(f"    Artist source breakdown:")
    for src in ["direct", "global", "none"]:
        c = src_artist.get(src, 0)
        print(f"      {src:>11}: {c:>7,}  ({100*c/total:.1f}%)")

    return results, dict(src_album)


# ======================================================================
# 10. Main
# ======================================================================


def main():
    # ================================================================
    # Load all data
    # ================================================================
    user_ratings = parse_training(TRAIN_FILE)
    track_meta = parse_tracks(TRACK_FILE)
    album_meta = parse_albums(ALBUM_FILE)
    test_users = parse_test(TEST_FILE)

    # ================================================================
    # Enrich tracks with album metadata
    # ================================================================
    print("\n" + "=" * 60)
    print("  ENRICHING TRACKS WITH ALBUM METADATA")
    print("=" * 60)
    enriched_meta = enrich_tracks_with_albums(track_meta, album_meta)

    # ================================================================
    # Build cold-start structures (on enriched metadata)
    # ================================================================
    print("\n" + "=" * 60)
```

```
    print("  PART 2: Building Fallback Structures (Enriched)")
    print("=" * 60)

    album_to_tracks = build_album_to_tracks(enriched_meta)
    global_stats = build_global_stats(user_ratings)

    # ================================================================
    # Part 2a: Global Fallback Only (no Dig Deeper)
    # ================================================================
    print("\n" + "-" * 60)
    print("  Part 2.a: Global Fallback Only")
    print("-" * 60)

    res_global_wa, _ = run_strategy_v2(
        user_ratings, enriched_meta, test_users,
        strategy_weighted_avg,
        album_to_tracks, global_stats,
        os.path.join(OUTPUT_DIR, "submission_p2a_global_weightedavg.csv"),
        use_dig_deeper=False, use_global=True,
    )

    res_global_mg, _ = run_strategy_v2(
        user_ratings, enriched_meta, test_users,
        strategy_max_genre,
        album_to_tracks, global_stats,
        os.path.join(OUTPUT_DIR, "submission_p2a_global_maxgenre.csv"),
        use_dig_deeper=False, use_global=True,
    )

    # ================================================================
    # Part 2b: Full Hierarchical (Direct -> Dig Deeper -> Global)
    # ================================================================
    print("\n" + "-" * 60)
    print("  Part 2.b: Full Hierarchical (Dig Deeper + Global)")
    print("-" * 60)

    res_hier_wa, src_hier_wa = run_strategy_v2(
        user_ratings, enriched_meta, test_users,
        strategy_weighted_avg,
        album_to_tracks, global_stats,
        os.path.join(OUTPUT_DIR, "submission_p2b_hier_weightedavg.csv"),
        use_dig_deeper=True, use_global=True,
    )

    res_hier_mg, src_hier_mg = run_strategy_v2(
        user_ratings, enriched_meta, test_users,
        strategy_max_genre,
```

```
        album_to_tracks, global_stats,
        os.path.join(OUTPUT_DIR, "submission_p2b_hier_maxgenre.csv"),
        use_dig_deeper=True, use_global=True,
    )

    res_hier_ev, _ = run_strategy_v2(
        user_ratings, enriched_meta, test_users,
        strategy_evidence_weighted,
        album_to_tracks, global_stats,
        os.path.join(OUTPUT_DIR, "submission_p2b_hier_evidence.csv"),
        use_dig_deeper=True, use_global=True,
    )

    # ================================================================
    # Impact Analysis
    # ================================================================
    print("\n" + "=" * 60)
    print("  IMPACT ANALYSIS")
    print("=" * 60)

    total = len(res_global_wa)

    # Global vs Hierarchical (WeightedAvg)
    changed_g_h_wa = sum(1 for (_, a), (_, b)
                         in zip(res_global_wa, res_hier_wa) if a != b)
    print(f"\n  WeightedAvg: Global vs Hierarchical:  "
          f"{changed_g_h_wa:,} changed ({100*changed_g_h_wa/total:.1f}%)")

    # Global vs Hierarchical (MaxGenre)
    changed_g_h_mg = sum(1 for (_, a), (_, b)
                         in zip(res_global_mg, res_hier_mg) if a != b)
    print(f"  MaxGenre:    Global vs Hierarchical:  "
          f"{changed_g_h_mg:,} changed ({100*changed_g_h_mg/total:.1f}%)")

    # Dig Deeper trigger rates
    print(f"\n  Dig-Deeper Trigger Rate (Hierarchical):")
    dd_wa = src_hier_wa.get("dig_deeper", 0)
    dd_mg = src_hier_mg.get("dig_deeper", 0)
    print(f"    WeightedAvg: {dd_wa:,} / {total:,}  "
          f"({100*dd_wa/total:.1f}%)")
    print(f"    MaxGenre:    {dd_mg:,} / {total:,}  "
          f"({100*dd_mg/total:.1f}%)")

    # Cross-strategy agreement (Part 2b)
    print(f"\n{'='*60}")
    print("  Part 2b Cross-Strategy Agreement")
    print(f"{'='*60}")
```

```python
    for na, nb, ra, rb in [
        ("Hier+MaxGenre",    "Hier+WeightedAvg", res_hier_mg, res_hier_wa),
        ("Hier+MaxGenre",    "Hier+Evidence",    res_hier_mg, res_hier_ev),
        ("Hier+WeightedAvg", "Hier+Evidence",    res_hier_wa, res_hier_ev),
    ]:
        agree = sum(1 for (_, a), (_, b) in zip(ra, rb) if a == b)
        print(f"  {na:>18} vs {nb:<18}:  "
              f"{agree:,}/{total:,}  ({100*agree/total:.1f}%)")

    # ---- Final summary ----
    print(f"\n{'='*60}")
    print("  PART 2 SUBMISSION FILES")
    print(f"{'='*60}")
    subs = [
        ("Part 2a -- Global+WeightedAvg",
         "submission_p2a_global_weightedavg.csv"),
        ("Part 2a -- Global+MaxGenre",
         "submission_p2a_global_maxgenre.csv"),
        ("Part 2b -- Hier+WeightedAvg",
         "submission_p2b_hier_weightedavg.csv"),
        ("Part 2b -- Hier+MaxGenre",
         "submission_p2b_hier_maxgenre.csv"),
        ("Part 2b -- Hier+Evidence",
         "submission_p2b_hier_evidence.csv"),
    ]
    for label, fname in subs:
        print(f"  {label:<35} {fname}")

    print("\nDone. All files in:", OUTPUT_DIR)


if __name__ == "__main__":
    main()
```

Listing 2: EE627_Midterm_part2_v2.py — Part 2 complete source code.